Neurolabs Cordova SDK Integration Guide

Technical Integration Specification

Document ID: NLB-CORDOVA-INTEGRATION-GUIDE
Version: v1.7.21
Date: 2026-10-16
Status: Partner integration reference
Owner: Neurolabs

1. Scope

This guide is for partner hybrid apps integrating ‘neurolabs-cordova-sdk‘.

2. Changes

The Cordova plugin is a thin bridge over the native SDKs, so most changes listed here come from the underlying ‘neurolabs-android-sdk‘ and ‘neurolabs-ios-sdk‘ releases that the plugin pulls via ‘scripts/prepare-sdk.js‘ at install time. No public JS API has been broken since v1.1.9.

v1.7.21

- Plugin pins the native SDKs’ 1.7.21 on both platforms.
- Store context and visit KPIs, on both platforms (see 6d):

- ‘prefetchStoreContext‘, ‘getStoreContextStatus‘, ‘setStoreContextIdentity‘ and ‘clearStoreContextCache‘ fetch, check and cache each outlet’s targets; - ‘setVisitKpisEnabled‘ and ‘setVisitKpiContext‘ control the visit KPIs; - the new ‘visitKpis‘ event delivers a finished session’s on-device KPIs and next best actions.

- The camera refuses a photo the queue would refuse (‘QUEUE_FULL‘, ‘STORAGE_LOW‘) before the shutter on both platforms, and says why on screen. ‘uploadFailed‘ then has no ‘captureId‘, and there is no ‘captureResult‘ or ‘captureQueued‘ for that photo (until 1.7.20, iOS took the photo and reported both).
- A camera that cannot run closes once with ‘cameraClosed.error‘ on both platforms: ‘CAMERA_PERMISSION_DENIED‘, ‘CAMERA_UNAVAILABLE‘ (new) or ‘CAMERA_PRESENT_FAILED‘ (iOS, new). It is never an ‘uploadFailed‘. On iOS, reopening the camera from the ‘cameraClosed‘ handler no longer wedges ‘openCamera‘.
- ‘captureResult.quality‘ is the quality verdict the photo was admitted with, in the same shape on both platforms.
- ‘flushQueue({ force: true })‘ re-sends failed photos that ‘retryFailedUploads()‘ leaves failed (see 5).
- ‘validationPreset: ’minimal’‘ now reaches the native minimal preset. With both overlays off and no ‘validationPreset‘, both platforms use lenient validation.
- Android: the toolchain check tests the real requirements (AGP 8.9.1, Gradle 8.11.1, compileSdk 36, minSdk 26, Kotlin 2.2, JDK 17), so cordova-android 14 works once ‘config.xml‘ meets them (see 3).
- Native: the image-quality checks behave the same on iOS and Android, and capture no longer waits 5 s on "Processing capture..." while the detector warms.

v1.7.20

- Plugin pins the native SDKs' 1.7.20 on both platforms. It ships the REX/Smollan audit fixes; 1.7.19 is unchanged, so upgrade to 1.7.20 to get them.
- iOS: the plugin adds the background-upload wake method to 'CDVAppDelegate' at launch, only if none is there and never replacing one. Plugin 1.7.19 declared it at compile time, which clashed with another plugin declaring the same method (see 5, item 23).
- Android: the plugin stops early, with one clear message, when cordova-android is older than 15, the Android Gradle Plugin older than 8.9.1 or the JDK older than 17 (see 3, item 24). cordova-android 14 is no longer accepted.
- Subscribing with 'addListener' no longer delivers a fake event at once (for example a 'cameraClosed' the moment you subscribe), on both platforms. A host that filtered it out ('status === "listening"') can drop that check.
- Native: the default camera tells the rep why it refused a photo for low storage or a full queue. On Android the photo is not taken, and 'uploadFailed' comes without a 'captureId' and with no failed 'captureResult'; iOS takes it and reports both, as before.
- Native: 'retryFailedUploads()' leaves a mission refusal that cannot succeed as sent failed, and a call within 60 s of the previous one does nothing (see 5). On Android, 'clearUploadQueue()' also drops the uploaded-but-unsubmitted photos' submits, as on iOS.
- Native: "Preparing detector..." holds the shutter for 3 s at most, and SDK error reports to Sentry never carry a server response body.
- iOS: with Wi-Fi-only uploads on, a background transfer never uses cellular data; photos go to iOS's background upload only from the foreground, and the background-wake completion handler is always called.

v1.7.19

- Plugin pins the native SDKs' 1.7.19 on both platforms.
- iOS: queued photos keep uploading through an iOS background 'URLSession' after iOS suspends or terminates the app and while the phone is locked. The plugin hands iOS's background wake to the SDK, so no native code or 'Info.plist' change is needed; a host 'AppDelegate' that implements 'application(_:handleEventsForBackgroundURLSession:completionHandler:)' itself forwards one line (see 5).
- New 'config.xml' preference 'NeurolabsSentryEnabled': 'value="false"' turns the SDK's Sentry off from app launch, before any JavaScript runs, and wins over 'init({ sentryEnabled })' (item 22).
- Native: Sentry reports only the SDK's own errors and crashes in SDK code, on both platforms. No host crashes, app hangs, watchdog terminations or failed requests, no host instrumentation, scrubbed URLs, and the iOS crash handler never replaces your app's own Sentry; 'sentryEnabled: false' also closes it (item 22).

- Native: on "Review shelf", a broken bay link offers "Retake bay N", which replaces that bay in place and merges the walk again, and the submit's ‘device_dedup‘ adds the seams between segments. A walk that folds back across a broken link to a bay it already took no longer counts it twice, and the submit adds the bays' left-to-right shelf order and their links. iOS: a retaken bay never merges with the next bay. Android: bays keep the order they were taken in through the merge and the submit.
- Native: how long on-device recognition keeps using its last good build while offline is set per organisation by Neurolabs (1 to 30 days, default 7). Until the backend setting ships, every organisation keeps 7 days.
- Native: a negative ‘device_dedup‘ count is never sent; on iOS a slow save no longer loses its photos, and on Android an accepted photo is no longer lost when its session folder was pruned.

v1.7.18

- Plugin pins the native SDKs' 1.7.18 on both platforms.
- Native: each capture's metadata record (capture contract v1: shutter time, device, camera, image size, attitude, intrinsics, an EXIF subset, the AR pose) is uploaded after its image, off the image path; a metadata failure never holds up the image or the submit. The detections go in it only with ‘sendDetectionsMetadata: true‘ (item 19).
- Native: a multi-bay walk with a pair of bays the SDK could not link no longer leaves the rest of the walk unmerged: the dedup runs inside each linked segment, "Review shelf" marks the broken link ("Couldn't link bay N - retake it to join", with "Open bay N"), and the submit also sends the walk's stitched realogram (shelves, rows and slots). ‘dedupTrusted‘ is still true only for one trusted segment.
- Native: with on-device recognition off (‘recognition: false‘), the AR boxes keep their identity from appearance as well as position (a colour histogram, or the ConvNeXt model for an organisation Operations grants it to). It is on by default and changes the overlay only, not counts or uploads.
- Android: ‘enablePreviewCropping: true‘ is display only, as on iOS: uploads carry the full frame, and only the thumbnail is trimmed (item 18). With ‘sendDetectionsMetadata: false‘, ‘captureResult.detections‘ is no longer empty, and a multi-bay session's ‘countsByLabel‘ and submitted ‘device_dedup‘ counts come from the real detections (they were empty before).
- iOS: an upload in flight when the app goes to the background gets iOS's grace period (about 30 s), and ‘init({ analyticsEnabled: false })‘ now also stops the SDK camera's analytics events (Android already did).
- New ‘init({ captureGeolocation, captureDepth })‘ and ‘captureResult.captureMetadata‘: each capture's metadata (capture contract v1) is uploaded after its image and handed to the host. Location and depth are opt-in, and the host app owns the location permission (see 4a).
- New ‘openCamera({ arHighResolutionStills: true })‘, Android only and experimental: full-resolution AR photos through ARCore's Shared Camera. iOS accepts the key and ignores it (see 6b).
- New ‘openCamera({ arAppearanceWithoutRecognition })‘ (default on): AR box identity from appearance with on-device recognition off (see 6b).

- ‘cameraClosed.multiBay‘ adds ‘realogram‘ (the walk’s shelves, rows and slots) and ‘segments‘ (its dedup segments), the same shape on both platforms (see 6a).

v1.7.17

- Plugin pins the native SDKs’ 1.7.17 on both platforms.
- Native: the default and AR cameras show an offline chip while there is no network ("Offline", plus "uploads queued" when photos wait), or "Waiting for Wi-Fi" while Wi-Fi-only uploads hold photos on cellular. It is on by default and never blocks the shutter; ‘openCamera({ showsOfflineIndicator: false })‘ hides it (see 6).
- Native: the AR flow’s "Review shelf" screen adds "On this shelf": the walk’s brands, share of shelf and SKUs from on-device recognition, or the total only when recognition is unavailable on the device.
- A continuous AR sweep (‘multiBayAR‘ preset, or ‘cameraMode: ’ar’‘ with ‘captureMode: ’continuous’‘) may plan up to 30 bays through the plugin on both platforms; every other session keeps 8 (see 6a).
- New ‘openCamera({ arBoxUnmatchedColor })‘: under ‘arBoxStyle: ’highlight’‘ with live labels on, a box the labels have looked at and not matched is drawn in this colour (default ‘#FACC15‘; see 6b).
- ‘arLiveLabels: true‘ now shows product names through the plugin on Android and iOS (needs recognition on and an operations-capable key; see 6b).

v1.7.16

- Plugin pins the native SDKs’ 1.7.16 on both platforms.
- New ‘init({ detectorPrefetchPolicy: ’unmetered’ })‘: the detector downloads only on Wi-Fi or Ethernet (native SDK 1.7.16). While it waits, ‘getDetectorModelStatus()‘ reports ‘reason: "waiting_for_unmetered_network"‘.
- New ‘init({ recognition: false })‘: the plugin never builds on-device recognition.
- New section 6c, "Capture and quality checks only (no boxes, no recognition)", with an upgrade checklist from plugin 1.6.6.
- New ‘openCamera({ finalReview: true })‘: an end-of-session review with tap-to-crop on the default camera (native SDK 1.7.16; see 6).
- New AR overlay options ‘arBoxStyle‘, ‘arBoxHighlightColor‘, ‘arCoverageStyle‘, ‘arCoverageOutlineColor‘, ‘arBoxPersistence‘, and the experimental ‘arLiveLabels‘ / ‘arLiveCount‘ (native SDK 1.7.16; see 6b). Live labels need recognition on and the catalog synced, and show names on Android, and on iOS from native 1.7.17 on.
- New ‘init({ maxRetainedPhotos, retainedPhotoExpiryDays })‘, the ‘retainedPhotosHigh‘ event, ‘retainedCount‘ / ‘retainedBytes‘ in the queue status, and the ‘STORAGE_LOW‘ capture refusal (native SDK 1.7.16; see 5 and 7). Uploaded photos kept on the device no longer count toward ‘maxQueueSize‘.
- Capture size: the plugin no longer sends 1920 px / 0.85 when you omit ‘maxImageDimension‘ / ‘imageCompressionQuality‘. The SDK default is now 4032 px at JPEG 0.9 on both platforms, and Neurolabs can override it per organisation (see 6). Set the values yourself only if upload size matters more than recognition quality.
- ‘validationPreset: "lenient"‘ now applies the native lenient thresholds (it used to apply the default ones), so fewer captures are refused.

- Native: quality checks run on the image alone until the detector is ready; the camera says why the detector is unavailable; uploaded photos never fill the upload queue; AR multi-bay handles right-to-left walks and shows the whole photo in the preview.

v1.7.15

- Plugin pins the native SDKs' 1.7.15 on both platforms.
- AR multi-bay (‘preset: "multiBayAR"‘): no per-bay preview; the bay is committed at the shutter and "Review shelf" after Finish is the one review (tap a bay to open it, crop, Previous / Next). Guidance holds while AR tracking is limited. New ‘openCamera({ arFlowShowsCapturePreview: true })‘ brings the per-bay preview back (also needs ‘showCapturePreview‘; see 6b).
- iOS: the downloaded detector and embedder now install on a physical device (they never compiled on a phone with native 1.7.13 / 1.7.14).
- Facings and recognition shares count SKU boxes only, at or above the 0.5 match floor: counts may drop.

v1.7.14

- Plugin pins the native SDKs' 1.7.14 on both platforms. No JS API changes.
- ‘detectorModelStatusChanged‘ / ‘getDetectorModelStatus()‘ can report two more ‘reason‘ codes: ‘insufficient_storage‘ (free space, then the SDK retries straight away) and ‘load_failed‘ (retried on the next launch).
- Native: on-device recognition keeps working offline from the last good sync; mission submits are idempotent; each tenant's upload queue stays separate across re-init.

v1.7.13

- Plugin pins the native SDKs' 1.7.13 on both platforms.
- The native SDKs bundle no detector: ‘all_in_one‘ downloads on first use, so the first launch needs network for live detection (capture and uploads work without it). The package is about 100 MB smaller per platform.
- New ‘getDetectorModelStatus()‘, ‘prefetchDetectorModel()‘ and ‘detectorModelStatusChanged‘, with one failure ‘reason‘ vocabulary on both platforms.
- ‘yoloModelVariant‘: ‘all_in_one‘ is the only detector; the legacy names are deprecated aliases for it.
- Re-init: ‘init‘ with another configuration now applies it, on a fresh native core. The effective operations key (‘operationsApiKey‘, else ‘apiKey‘) selects the upload queue, so each tenant's queued captures stay with that tenant. ‘init‘ without ‘operationsBaseUrl‘ resets the operations base to production instead of keeping a base set by an earlier ‘init‘.
- Boolean ‘init‘ options are read the same way on both platforms: ‘true‘/‘false‘, a number (0 is false), or ‘"true"‘/‘"false"‘, ‘"1"‘/‘"0"‘, ‘"yes"‘/‘"no"‘, ‘"on"‘/‘"off"‘. Android used to ignore numbers and the word forms. For example, ‘uploadsEnabled: "no"‘ or ‘0‘ now disables uploads there.

v1.7.12

- Plugin pins the native SDKs' 1.7.12 on both platforms.
- iOS: distance guidance ("move closer / back off") works on the AR camera.

- Android: failed uploads retry on foreground and network return; the shelf overlay follows the camera.
- Bridge: argument checks are the same on both platforms (stricter rule); see the CHANGELOG for the behaviour changes.

v1.7.11

- Plugin pins the native SDKs' 1.7.11 on both platforms.
- Security (iOS bridge): the previous tenant's recognition provider is cleared on a credential change or a re-init without a key.
- A whitespace-only 'apiKey' is rejected on iOS, as on Android.
- Android: the shelf overlay and live boxes sit on the products in portrait.
- iOS: two crashes fixed (extreme remote-config values; repeated 'issuePriority' kinds).

v1.7.10

- Plugin pins the native SDKs' 1.7.10 on both platforms.
- Captures are portrait-only again: native 1.7.9's landscape orientation is switched off because the backend pipelines assume portrait images.
- 'openCamera' rejects a 'customCameraTemplate' that is not an object with 'INVALID_ARGUMENT'; one bad template field no longer discards the whole template on iOS.
- Android 'getPhoto' / 'deletePhoto' reject with 'INTERNAL_ERROR' instead of never settling when the queue read fails.

v1.7.9

- Plugin pins the native SDKs' 1.7.9 on both platforms.
- New 'openCamera' options for AR capture, forwarded on both platforms: 'captureMode: 'guided' | 'continuous'' and the AR multi-bay preset ('preset: 'multiBayAR'' or 'multiBayAR: {...}'). See section 6b.
- AR detection boxes are always world-locked. There is no renderer option: a leftover 'arBoxRenderer' key is ignored.
- New 'getARCapabilities()' returning '{ worldTrackingSupported, depthSupported, installRequired }'.
- Android: ARCore dependency 1.52.0 -> 1.56.0.

v1.3.2

- Plugin pulls native SDK v1.3.2 binaries on 'cordova plugin add': Android AAR + iOS xcframework refreshed via the mobile-dist manifests.
- Native v1.3.2 highlights (no JS surface change required):

- Mixpanel ingestion now lands in the EU region (events were previously POSTed to the US endpoint and silently dropped). iOS also fixes a separate bug where Mixpanel rejected every event because ‘time‘ was serialized as a string rather than a JSON number. - Done button stays responsive while the SDK persists captures - disk I/O moved off the main thread on both platforms, inline spinner / progress indicator surfaced. - Android: native ‘SIGSEGV‘ fix in ‘com.google.ai.edge.litert.Model.nativeLoadAsset‘ when the TFLite asset is stored compressed in the APK; the loader probes the asset with ‘AssetManager.openFd‘ first and extracts to internal storage when the asset is not mmap-able. - Onboarding barcode pipeline normalises to GTIN-14 end-to-end and propagates the scanned barcode format through every product-lookup client. - iOS: new ‘NLOperationsCatalogClient‘ (in ‘ProductAuditKit‘) for the operations catalog; replaces ‘precondition‘-based validation with thrown errors so a malformed barcode no longer crashes the host app.

v1.2.5

- Native sequence-counter parity: Cordova bridge surfaces the ‘showsSequenceCounterLabel‘ (and ‘showsCapturedCountLabel‘) options to match the native SDK toggles.
- Demo SDK setup builds fixed; iOS Sentry injection timing hardened; Android LiteRT dependency pinning correction; sequence-counter argument plumbed through correctly.

v1.2.3

- ‘openCamera‘ supports ‘cameraMode: "ar" | "default"‘ - ‘"default"‘ enables the 0.5/1 lens switcher on both iOS and Android.
- ‘init‘ supports ‘analyticsEnabled‘, ‘sentryEnabled‘, ‘sentryDsn‘ for controlling SDK analytics and crash reporting.
- Tightened default quality validation thresholds (blur, glare, perspective) across both platforms.

v1.2.2

- Internal: Android plugin integration handling refactor; no public JS API change.

v1.2.1

- Cordova install hooks hardened; demo integration updated.

v1.2.0

- Internal scaffolding for the v1.2.x line; no JS API change.

v1.1.9

- Init and per-session routing use ‘taskUUID‘.
- Native detector/model warmup is enforced in ‘init‘ and guarded in ‘openCamera‘.
- ‘openCamera‘ can return ‘MODEL_INIT_FAILED‘ when native model initialization fails.
- ‘autoCloseAfterCapture=true‘ now keeps preview enabled and closes after Save confirmation.
- Manual finish (‘Done‘) is available when ‘autoCloseAfterCapture=false‘ and ‘allowManualFinish=true‘.
- ‘cameraClosed‘ event now includes ‘message‘.
- ‘captureQueued‘ events are deferred while camera is open and flushed after ‘cameraClosed‘.

3. Install

Download the plugin package from ‘https://github.com/neurolaboratories/neurolabs-mobile-dist/releases‘. The install hooks download and wire up the native SDKs automatically - no manual AAR or xcframework placement needed.

Native baseline requirements:

- Android: Android Gradle Plugin 8.9.1+ (the SDK’s androidx dependencies require it) with Gradle 8.11.1+ (AGP 8.9.1’s minimum), ‘compileSdkVersion 36‘, ‘minSdkVersion 26‘, JDK 17+, Kotlin Gradle plugin 2.2+. Any cordova-android that builds with these works, 14 included. cordova-android 15’s defaults (AGP 8.10.1, Gradle 8.14.2, SDK 36) meet all but minSdk (24); cordova-android 14’s (AGP 8.7.3, Gradle 8.13, SDK 35, minSdk 24) need the ‘config.xml‘ preferences below. The plugin injects ‘GradlePluginKotlinVersion 2.3.0‘ and raises the Java source/target compatibility to 17 on its own; override the Kotlin version only via a ‘<platform name="android">‘-scoped preference, and only to 2.2 or later.
- iOS: deployment target iOS 17.0+, Xcode 16.4+ recommended (release toolchain baseline).

Android toolchain check (plugin 1.7.20+). Before the plugin is installed and before every Android prepare or build, a plugin hook checks the Android Gradle Plugin, Gradle, compileSdk, minSdk, the Kotlin Gradle plugin and the JDK. If any of them is too old, it stops with one message that names each unmet requirement, what it found and the plugin version. Without it, the build fails much later with a Kotlin metadata, manifest-merger or Gradle error. Plugin 1.7.20 also required cordova-android 15; from 1.7.21 the cordova-android major is not checked. From 1.7.21 the check runs before the install, so a failed check installs nothing: fix ‘config.xml‘ and run ‘cordova plugin add‘ again. (Plugin 1.7.20 checked after the install; after a failure there, run ‘cordova plugin rm ai.neurolabs.cordova‘ before retrying.)

The hook reads ‘config.xml‘ the way cordova-android does (the Android-scoped preference wins, the last duplicate wins), each else the installed cordova-android’s default: ‘AndroidGradlePluginVersion‘; ‘GradleVersion‘; ‘android-compileSdkVersion‘, else ‘android-targetSdkVersion‘; ‘android-minSdkVersion‘; ‘GradlePluginKotlinVersion‘, else the plugin’s injected 2.3.0. Where ‘config.xml‘ sets none, a value another installed plugin injects into ‘config.xml‘ (recorded in ‘platforms/android/android.json‘) applies first, as in cordova-android’s merged config; if two plugins inject different values, that check is skipped. The JDK comes from ‘javac -version‘ (‘CORDOVA_JAVA_HOME‘, ‘JAVA_HOME‘ or ‘PATH‘). It skips a value something else may override: Gradle under ‘CORDOVA_ANDROID_GRADLE_DISTRIBUTION_URL‘, and an SDK level under a ‘cdvCompileSdkVersion‘ or ‘cdvMinSdkVersion‘ Gradle property (‘--gradleArg=-P...‘, ‘ORG_GRADLE_PROJECT_*‘, a ‘gradle.properties‘). It skips a value it can’t read, and iOS-only projects never run it. ‘NEUROLABS_SKIP_ANDROID_TOOLCHAIN_CHECK=1‘ turns the failure into a warning; that setup isn’t supported.

Add to ‘config.xml‘ on cordova-android 15 (set ‘AndroidGradlePluginVersion‘ only to 8.9.1 or later):

```
<platform name="android">
    <preference name="android-minSdkVersion" value="26" />
    <preference name="android-compileSdkVersion" value="36" />
</platform>
```

On cordova-android 14, also raise AGP (its default Gradle, 8.13, already meets 8.11.1):

```
<platform name="android">
    <preference name="AndroidGradlePluginVersion" value="8.9.1" />
    <preference name="android-minSdkVersion" value="26" />
    <preference name="android-compileSdkVersion" value="36" />
</platform>
```

Android (Windows or macOS)

```
# Add the Android platform first, then install the plugin
cordova platform add android
cordova plugin add file:/absolute/path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

Give the ‘.tgz‘ as a ‘file:‘ spec with an absolute path. Cordova CLI 12 and 13 hand a bare path to npm, which fails with ‘E404‘.

The ‘before_plugin_install‘ hook downloads ‘neurolabs-android-sdk.aar‘ from the matching GitHub release, stores it inside the plugin, and ‘plugin.xml‘ copies it to ‘app/libs/‘ automatically. ‘neurolabs.gradle‘ wires it into the build via ‘flatDir‘.

iOS (macOS only)

```
# Add the iOS platform first, then install the plugin
cordova platform add ios
cordova plugin add file:/absolute/path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

The hook downloads ‘NeurolabsSDK.xcframework‘ from the matching GitHub release and Cordova embeds it automatically during build. Requires ‘curl‘ (standard on macOS) or ‘gh‘ CLI. For private releases set ‘GITHUB_TOKEN‘.

The iOS install hook injects the ‘Sentry‘ Swift Package dependency into the generated Xcode project automatically during platform preparation. If Xcode still reports ‘unable to find module dependency: Sentry‘, re-run ‘cordova prepare ios‘ so the package graph refreshes in the generated project.

Optional overrides (use a pre-downloaded file or a specific URL instead of auto-download):

```
# Local file
NEUROLABS_IOS_XCFRAMEWORK_ZIP=/path/to/NeurolabsSDK.xcframework-vX.Y.Z.zip \
  cordova plugin add file:/absolute/path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

NEUROLABS_ANDROID_AAR_PATH=/path/to/neurolabs-android-sdk-vX.Y.Z.aar \
  cordova plugin add file:/absolute/path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

# Direct URL
NEUROLABS_IOS_XCFRAMEWORK_URL=https://github.com/.../NeurolabsSDK.xcframework-vX.Y.Z.zip \
  cordova plugin add file:/absolute/path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

NEUROLABS_ANDROID_AAR_URL=https://github.com/.../neurolabs-android-sdk-vX.Y.Z.aar \
  cordova plugin add file:/absolute/path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

iOS + Android (macOS)

Add both platforms before installing the plugin - both hooks run in a single ‘cordova plugin add‘.

4. SDK Initialization + Warmup

```
const Neurolabs = cordova.require('ai.neurolabs.cordova.Neurolabs');

await Neurolabs.init({
  apiKey: '<API_KEY>',
  operationsApiKey: '<OPERATIONS_API_KEY>', // REQUIRED for uploads (see note below)
  // operationsBaseUrl: 'https://api.operations.staging.neurolabs.ai/v1', // optional overrid
e (staging / self-hosted)
  taskUUID: '<DEFAULT_TASK_UUID>',
  allowBase64PhotoExport: false,
  returnCaptureFileUris: false,    // v1.6.5 opt-in file-URI streaming (see 8.1)
  analyticsEnabled: true,          // SDK analytics (default true)
  sentryEnabled: true,             // Sentry crash reporting (default true)
  // maxRetainedPhotos: 500,       // kept uploaded photos before retainedPhotosHigh warns (s
ee 5)
  // retainedPhotoExpiryDays: 14,  // delete kept uploaded photos after N days (default off)
  // captureGeolocation: false,    // location in each capture's metadata; the app owns the p
ermission (see 4a)
  // captureDepth: false,          // AR depth map with each AR capture (see 4a)
  // sentryDsn: 'https://...',     // optional custom Sentry DSN
  // yoloModelVariant: 'all_in_one', // the only detector; see below
});
```

> **Upgrade note (pre-1.6 integrators): ‘operationsApiKey‘ is now required for uploads.** > > The legacy ‘/images‘ upload endpoint has been removed - the native > capture queue is operations-only. If you call ‘init()‘ **without** > ‘operationsApiKey‘, ‘init()‘ and ‘openCamera()‘ still succeed and photos > still queue, but **every upload fails silently**: the queue item errors > with ‘OPERATIONS_KEY_REQUIRED‘ (non-retryable), surfaced only through the > ‘uploadFailed‘ event. There is no console error and no init-time > rejection, so this is easy to miss during an upgrade. > > “‘js > Neurolabs.addListener('uploadFailed', ({ errorCode, message }) => { > if (errorCode === 'OPERATIONS_KEY_REQUIRED') { > console.error('Missing operationsApiKey in init() - uploads will never succeed.', message); > } > }); > “‘ > > ‘operationsBaseUrl‘ (optional) overrides the operations endpoint for > staging / self-hosted deployments. It must be an ‘https‘ URL with a host > and no embedded credentials; ‘http‘, malformed, or userinfo-bearing > values reject at ‘init()‘ with ‘INVALID_ARGUMENT‘. Omit it to keep the > production default (‘https://api.operations.neurolabs.ai/v1‘).

Detector model (downloaded at runtime)

Since native SDK 1.7.13 the SDK bundles **no** detector model. Its one detector, ‘all_in_one‘, is downloaded on first use through the Operations API with the same operations credential as uploads, verified against a pinned sha256 and cached on the device. The plugin package is about 100 MB smaller per platform as a result.

- **The first launch needs network for detection.** Until the model is on the device, capture, the shutter, the upload queue and the pixel quality checks (light, glare, blur) all work; live detection, the product overlay and detection-based guidance do not, and the native camera shows a "Detector downloading. You can still capture." chip. When the download lands, detection switches on mid-session with no restart.

- **Loading progress.** Without the model the SDK reaches ‘loadingProgress.step === "ready"‘ ("Ready (detector downloading...)") and never emits ‘"model_ready"‘. ‘waitUntilReady()‘ resolves on ‘"ready"‘; never wait for ‘"model_ready"‘ alone.
- **Status.** ‘Neurolabs.getDetectorModelStatus()‘ resolves ‘{ state, ready, reason? }‘ (‘state‘: ‘"not_downloaded" | "downloading" | "ready" | "failed"‘), and ‘detectorModelStatusChanged‘ fires on every change. ‘Neurolabs.prefetchDetectorModel()‘ (after ‘init‘) starts the download now, e.g. on Wi-Fi before the first capture; a failed download resolves with ‘state: "failed"‘ and a ‘reason‘, it does not reject. ‘reason‘ is one of ‘no_credentials‘, ‘not_entitled‘, ‘unknown_model‘, ‘pin_mismatch‘, ‘sha_mismatch‘, ‘download_failed‘, ‘api_unavailable‘, ‘compile_failed‘, ‘insufficient_storage‘, ‘load_failed‘ on both platforms; anything else arrives as ‘download_failed‘. The first five are retried after 1 h, the next three after 60 s; ‘insufficient_storage‘ (native SDK 1.7.14) is retried with no wait once space is freed (next capture session or ‘prefetchDetectorModel()‘), and ‘load_failed‘ (1.7.14) only after the app is relaunched.
- **Downloading only on Wi-Fi.** ‘init({ detectorPrefetchPolicy: ’unmetered’ })‘ (native SDK 1.7.16) holds the SDK’s own download until the device is on an unmetered network. Meanwhile the status is ‘not_downloaded‘ with ‘reason: "waiting_for_unmetered_network"‘, which is not a failure, and ‘prefetchDetectorModel()‘ still downloads at once. See 6c.
- **‘DETECTOR_UNAVAILABLE‘** (detection before the model lands) is expected and retryable. The plugin never reports it as ‘uploadFailed‘ or as an error.

‘yoloModelVariant‘ (optional) is now effectively fixed: its default and only value is ‘all_in_one‘. The legacy values ‘nlb_model‘, ‘yolo_v26‘, ‘yolo_v11_seg‘, ‘shelf_rows‘ and ‘yolo11l_seg_sam‘ are **deprecated** aliases of ‘all_in_one‘: still accepted, so existing configs keep working, but they no longer select a different model. An unknown value raises ‘INVALID_ARGUMENT‘.

Warmup is native-side; monitor progress through ‘loadingProgress‘ event.

```
Neurolabs.addListener('loadingProgress', (payload) => {
  console.log('loading', payload);
});
```

4a. Per-capture metadata, location and depth (native SDK 1.7.18)

From native SDK 1.7.18 the SDK camera records a metadata record for every photo it takes (capture contract v1): shutter time, device and OS, camera mode, image size, the detector’s boxes, an EXIF subset (never GPS or a serial number), the device’s attitude, the camera intrinsics and, on the AR camera, the camera pose. The record goes up after its image, off the image path: a metadata failure never holds up or fails the image or the mission submit. The detections go up only with ‘sendDetectionsMetadata: true‘.

The same record reaches your app as ‘captureResult.captureMetadata‘, in camelCase with epoch-millisecond times (the ‘CaptureMetadata‘ type in ‘types/index.d.ts‘). It is absent for a capture the SDK camera did not take.

Two ‘init‘ opt-ins add more, both off by default:

- ‘captureGeolocation: true‘ adds the device’s location (‘geolocation: { lat, lon, accuracyM, timestamp }‘, and ‘consent.geolocation: true‘). **Your app owns the location permission.** The plugin declares none and never asks for it. iOS: add ‘NSLocationWhenInUseUsageDescription‘ to the app’s Info.plist and request When In Use access before opening the camera. Android: declare ‘ACCESS_FINE_LOCATION‘ (or ‘ACCESS_COARSE_LOCATION‘) in the app’s manifest and request it at runtime. Without the permission, or without a fix, there is no location and no error. Location is sent only with your opt-in and the OS permission.
- ‘captureDepth: true‘ adds the AR camera’s depth map to each AR capture, uploaded as a side file and described by ‘depth: { depthId?, source, format, encoding, width, height, hasConfidence }‘. iOS: LiDAR scene depth, on LiDAR iPhones only (‘source: "lidar"‘). Android: the ARCore Depth API, depth from motion, on devices that support it (‘source: "arcore_depth"‘); it is noisier than LiDAR and has no confidence plane. A device without depth records none, without error.

Both are process-wide on the native side, so every ‘init‘ sets them: an omitted key turns the opt-in off. Anything but a boolean rejects with ‘INVALID_ARGUMENT‘. Coordinates, depth values and EXIF fields are never logged by the plugin or the SDKs. The backend endpoints that receive the record must be live on your Operations environment; until then the record is dropped quietly and the images upload as before.

```
await Neurolabs.init({ apiKey: '<API_KEY>', captureGeolocation: true });
Neurolabs.addListener('captureResult', ({ captureId, captureMetadata }) => {
  if (captureMetadata?.consent.geolocation) {
    const { lat, lon } = captureMetadata.geolocation ?? {};
    // store with your own record of captureId; do not log it
  }
});
```

5. Queue Management

```
await Neurolabs.setAutoSyncEnabled(true);
await Neurolabs.setWifiOnlyUploadsEnabled(false);
await Neurolabs.setDeletePhotosOnUploadSuccess(true);

await Neurolabs.pauseQueue();
await Neurolabs.resumeQueue();
await Neurolabs.setRetryPolicy({ retryCount: 5 });

const status = await Neurolabs.getQueueStatus();
console.log('queue status', status);

await Neurolabs.flushQueue();
await Neurolabs.retryFailedUploads();
// Explicit "Retry all": also re-sends terminal refusals.
await Neurolabs.flushQueue({ force: true });
```

From native 1.7.20, ‘retryFailedUploads()‘ gives failed photos a fresh cycle except a mission refusal that cannot succeed as sent (‘wrong_status‘, ‘submission_mismatch‘, ‘mission_not_found‘, ‘invalid_subsection_name‘, ‘invalid_subsection_id‘, or any ‘not_claimed_by_*‘), and a call within 60 s of the previous one does nothing, so a retry button or timer cannot reset every backoff.

To re-send those refusals too, call ‘flushQueue({ force: true })‘: it gives every failed photo a fresh cycle first, on iOS and Android. Use it for an explicit user action, not a timer. A failed photo keeps a ‘maxQueueSize‘ slot until it uploads or you delete it (‘deletePhoto‘, ‘clearUploadQueue‘), so a refusal you will not re-send is best deleted.

Uploads in the background and after the app is closed

- **Android:** the native queue keeps uploading while the app’s process lives, also in the background. The plugin initialises the SDK from JavaScript, so after the app is swiped away or killed, uploads resume the next time the app is opened and ‘init‘ runs.
- **iOS, from plugin 1.7.19 (native SDK 1.7.19):** each queued photo is uploaded through an iOS background ‘URLSession‘, so it keeps uploading after iOS suspends or terminates the app and while the phone is locked. The OS carries only the photo’s bytes to its upload URL, with no operations credential; ‘upload/complete‘ and the mission submit run in the app, on the background wake or the next foreground. A user force-quit cancels the transfers (iOS behaviour); nothing is lost, and the next launch resumes them. Native 1.7.18 gave an upload in flight only iOS’s grace period (about 30 s) after the app was backgrounded.
- **iOS background wake (no code needed).** When those uploads finish while the app is not running, iOS relaunches it in the background and calls the app delegate’s ‘application(_:handleEventsForBackgroundURLSession:completionHandler:)‘. From plugin 1.7.20, when the app starts, the plugin adds that method to ‘CDVAppDelegate‘, but only if ‘CDVAppDelegate‘ doesn’t already have one; it never replaces or swaps an existing method. It hands the wake to the SDK (‘NeurolabsSDKCore.handleEventsForBackgroundURLSession‘), and for a session that is not the SDK’s it calls the completion handler at once, exactly once. The method is added from an Objective-C ‘+load‘, which runs before ‘main‘ and so before UIKit creates the app delegate. That matters because a background relaunch connects no scene and loads no web view or plugin. Nothing goes in ‘Info.plist‘. Two cases need code in your app:

- **Your ‘AppDelegate‘ already implements this method** (your own background sessions, or another SDK’s). Yours wins over the plugin’s, so forward to the SDK first, then handle your own sessions: ‘if NeurolabsSDKCore.handleEventsForBackgroundURLSession(identifier, completionHandler: completionHandler) { return }‘ (‘import NeurolabsSDK‘). The plugin adds nothing at compile time, so your code compiles exactly as it does without the plugin. In Swift, cordova-ios itself requires ‘override‘ on this method: ‘CDVAppDelegate‘ adopts ‘UIApplicationDelegate‘, so Swift treats every delegate method as inherited. cordova-ios 8’s ‘@objc @implementation extension AppDelegate‘ accepts only methods declared in ‘AppDelegate.h‘, so put the method in a plain extension:

‘‘‘swift import NeurolabsSDK

extension AppDelegate { public override func application(_ application: UIApplication, handleEventsForBackgroundURLSession identifier: String, completionHandler: @escaping () -> Void) { if NeurolabsSDKCore.handleEventsForBackgroundURLSession(identifier, completionHandler: completionHandler) { return } // your own sessions; call completionHandler once for them completionHandler() } } ‘‘‘

An Objective-C ‘AppDelegate‘ handles its own sessions and passes any other identifier to the plugin’s method with ‘[super application:application handleEventsForBackgroundURLSession:identifier completionHandler:completionHandler];‘ (the SDK’s Swift API is not visible from Objective-C). - **Another plugin also handles this wake.** If it declares the method on ‘CDVAppDelegate‘ (an Objective-C category or a Swift extension), that method is already there at launch and this plugin adds nothing. That plugin then answers every wake, and the SDK’s never get forwarded. Implement the method in your ‘AppDelegate‘ as above: forward to the SDK first, then call ‘super‘ for the other plugin. Plugin 1.7.19 declared the method in a Swift extension, so another Swift extension with the same selector failed to compile and an Objective-C category made the winner undefined; upgrade to 1.7.20.

Without the wake handled, uploads still finish in the background; the SDK learns of them the next time the app runs.

- On both, queued photos survive a kill. Not every photo reaches the backend: a full queue or low storage refuses a new capture (‘QUEUE_FULL‘ / ‘STORAGE_LOW‘, reported as ‘uploadFailed‘), a terminal refusal or ‘clearUploadQueue()‘ removes a queued photo, and an already-uploaded photo kept on the device is deleted by ‘retainedPhotoExpiryDays‘ (see below).

What reaches the backend

- Per capture: the photo (the full frame, sized by ‘maxImageDimension‘ / ‘imageCompressionQuality‘) with its ‘taskUUID‘ and bay. Exception: on native Android up to 1.7.17 with ‘enablePreviewCropping: true‘, the uploaded photo was the detection trim; the full frame from native 1.7.18 (item 18).
- Per multi-bay session: the submit’s ‘device_dedup‘ summary (section 6a), and from native SDK 1.7.18 the walk’s stitched realogram (section 6a).
- Per capture, from native SDK 1.7.18: the capture’s metadata record, after the image (section 4a). Detections in it only with ‘sendDetectionsMetadata: true‘; location and depth only with their ‘init‘ opt-ins.
- Not uploaded: detections up to native 1.7.17 (‘sendDetectionsMetadata‘, item 19 in section 6c), the ‘enablePreviewCropping‘ trim (display only, except on native Android up to 1.7.17, item 18), and AR session recordings (‘.nlrec‘, native AR tooling, not a capture output).

Kept uploaded photos (native SDK 1.7.16)

With ‘deletePhotosOnUploadSuccess: false‘ (keep-photos mode), or while a session waits for its submit, the queue keeps uploaded photos on the device. From native SDK 1.7.16 they take **no** ‘maxQueueSize‘ slot: only pending, uploading and failed items do, so a host clean-up that lags no longer turns every new capture into ‘QUEUE_FULL‘.

- ‘init({ maxRetainedPhotos })‘ (whole number > 0, default 500) bounds them with a warning only. From 80% of it the ‘retainedPhotosHigh‘ event fires with ‘{ count, bytes, oldestCreatedAt?, limit }‘ (‘oldestCreatedAt‘ in epoch ms), once per crossing and again only after the count drops back below. It never refuses a capture.
- ‘init({ retainedPhotoExpiryDays })‘ (whole number > 0, or ‘null‘ / omitted for off, the default) deletes a kept photo, files included, captured at least that many days ago, at the next flush or launch. Photos kept only until their session’s submit never expire.

- Both are process-wide on the native side, so every ‘init‘ sets them; omitting one restores its default. Zero, a negative, a fraction, a boolean or a string rejects with ‘INVALID_ARGUMENT‘.
- ‘getQueueStatus()‘ and ‘queueStatusChanged‘ carry ‘retainedCount‘ and ‘retainedBytes‘.
- A new photo is refused with ‘STORAGE_LOW‘ (retryable) when the queue’s storage has under 500 MB free; it wins over ‘QUEUE_FULL‘. From native 1.7.21 the camera asks the queue before the shutter, so the photo is never taken. Both arrive through ‘uploadFailed‘ without a ‘captureId‘, the same way on iOS and Android (see 7).

```
await Neurolabs.init({ apiKey: ’<API_KEY>’, deletePhotosOnUploadSuccess: false, maxRetainedPh
otos: 300 });

Neurolabs.addListener(’retainedPhotosHigh’, async ({ count, bytes, limit }) => {
  console.warn(‘Kept ${count}/${limit} uploaded photos (${bytes} bytes); cleaning up’);
  await cleanUpHostCopies(); // e.g. Neurolabs.deletePhoto({ queueItemId }) for the oldest on
es
});
```

6. Open Custom Camera

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  cameraMode: 'default',          // 'ar' or 'default' - enables 0.5/1 lens switcher
  guidanceMode: 'strict',
  validationPreset: 'ios_parity',

  confidenceThreshold: 0.25,
  iouThreshold: 0.45,
  maxCaptures: 1,

  showAlignmentGuidance: true,
  enableValidation: true,
  liveQualityChecksEnabled: true,
  liveQualityTargetFps: 6,

  // keep custom-camera guidance UI path
  showDetections: false,
  showCapturedRegions: false,

  // optional metadata/cropping
  sendDetectionsMetadata: true,
  enablePreviewCropping: true,

  // now defaults to true if omitted, but explicit is clearer
  autoCloseAfterCapture: true,

  // manual-finish mode (Done button):
  // allowManualFinish: true,
  // minCapturesBeforeDone: 3,

  // per-session routing override
  taskUUID: 'bfc85982-b955-4f65-9f32-b6dbed85f364',

  // image size and quality: omit them unless upload size matters more than
  // recognition accuracy (see "Image size and quality" below)
  // maxImageDimension: 4032,
  // imageCompressionQuality: 0.9
});
```

End-of-session review (‘finalReview‘, native SDK 1.7.16)

A host that opens the default camera with ‘showCapturePreview: false‘ had no review at all: a rep could not crop or drop a bad shot. ‘finalReview: true‘ adds one review at the end of the session:

```
await Neurolabs.openCamera({
  cameraMode: 'default',
  showCapturePreview: false,
  autoCloseAfterCapture: false,
  allowManualFinish: true,
  finalReview: true,
});
```

- It applies to the default (non-AR) camera with no ‘multiBay‘; it is ignored on the AR camera. There is no per-shot preview, whatever ‘showCapturePreview‘ says.

- Done, or the count that ends the session today (‘maxCaptures‘, the auto-close target, or the required count when there is no Done button), opens **Review** with every photo. Tapping one opens it with Crop, Delete, Previous / Next and "Back to review".
- A crop replaces the photo under the same ‘captureId‘. A delete fires ‘captureDeleted‘ with ‘source: "review"‘.
- Nothing from the session is saved or queued before **Confirm**, so ‘captureQueued‘ arrives only after it. **Back to camera** keeps the photos and the shutter takes more; is the camera’s own close.

Offline chip (‘showsOfflineIndicator‘, native SDK 1.7.17)

Capture works offline, and the camera now says so. A small chip on the default and AR camera reads "Offline" when the device has no network, adds "uploads queued" after a middle dot when photos are waiting, and reads "Waiting for Wi-Fi" followed by "uploads queued" while Wi-Fi-only uploads hold photos on cellular. It never blocks the shutter, and it hides while the network state is unknown or uploads are off. It is on by default; ‘showsOfflineIndicator: false‘ hides it.

Image size and quality

‘maxImageDimension‘ (long edge, px), ‘maxImageWidth‘, ‘maxImageHeight‘ (px, ‘> 0‘) and ‘imageCompressionQuality‘ (JPEG, ‘0..1‘) are forwarded only when you set them; the plugin has no default of its own. The native SDK resolves each one in this order:

1. the value you pass to ‘openCamera‘; 2. your organisation’s remote config (‘sdkCapture.maxImageDimension‘ / ‘sdkCapture.imageCompressionQuality‘ in ‘GET /v1/recognition-config‘, set by Neurolabs per organisation); 3. the SDK default: a **4032 px long edge at JPEG quality 0.9** (native SDK 1.7.16).

The plugin pins native 1.7.21 (the default dates from 1.7.16), so on both platforms an omitted value means 4032 px at 0.9 unless your organisation overrides it. Invalid values (‘<= 0‘, or a quality outside ‘0..1‘) reject with ‘INVALID_ARGUMENT‘ on both platforms.

Lowering them costs recognition quality. The product crops, the price matcher and OCR all work on the uploaded photo, not on the live preview. A smaller or more compressed photo leaves fewer pixels in each product crop and price tag, so matching and price reading get worse. Set them only when upload size matters more than recognition accuracy.

Upgrade note. Earlier plugin releases sent ‘1920‘ / ‘0.85‘ whenever you omitted these options. The SDK read those as your own values, so they overrode both your organisation’s config and the SDK default. If you relied on that, you now get larger photos (4032 px at 0.9 from native 1.7.16) unless you set the values yourself or your organisation overrides them.

Manual-finish example:

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  guidanceMode: 'strict',
  maxCaptures: 10,
  autoCloseAfterCapture: false,
  allowManualFinish: true,
  minCapturesBeforeDone: 3
});
```

6a. Multi-Bay Capture (wide shelves)

Multi-bay capture (native SDK v1.6.0+) guides the user across several overlapping "bays" of a wide shelf in a single session and de-duplicates detections across them. Enable it by passing a ‘multiBay‘ options object to ‘openCamera‘:

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  guidanceMode: 'strict',
  multiBay: {
    minBays: 2,                  // 1..8 (1..30 for a continuous AR sweep)
    maxBays: 4,                  // 2..8 (2..30 for a continuous AR sweep)
    targetOverlapFraction: 0.30, // 0.05..0.9 (native default 0.30)
    onDeviceDedup: true,         // de-duplicate detections across bays on device
    generatePreview: true,       // build a stitched display-only preview
    previewMaxDimension: 2048    // max long-edge px of the stitched preview
  }
});
```

Out-of-range values reject with ‘INVALID_ARGUMENT‘, as does ‘minBays > maxBays‘. The ceiling depends on the session’s mode, the same on both platforms:

- **Continuous AR sweep: up to 30 bays.** A continuous sweep commits a bay on every auto-capture, so a shutter-free walk past several fixtures needs more than 8. The session counts as one when the effective ‘captureMode‘ is ‘’continuous’‘ and the camera is AR: the ‘multiBayAR‘ preset (section 6b, which always runs AR), or an explicit ‘cameraMode: ’ar’‘. There ‘minBays‘ takes ‘1..30‘ and ‘maxBays‘ ‘2..30‘.
- **Everything else: up to 8 bays.** Guided capture, ‘cameraMode: ’default’‘, and a continuous sweep with ‘cameraMode‘ omitted outside the preset (the two native SDKs pick a different camera when it is omitted, so the plugin does not assume AR). ‘9‘ or more rejects with ‘INVALID_ARGUMENT‘ (‘multiBay.maxBays must be between 2 and 8‘).

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  multiBayAR: { captureMode: 'continuous', multiBay: { maxBays: 20 } }
});
```

Which native SDK honours it: Android from native 1.7.16; iOS from native 1.7.17. With the native 1.7.16 iOS SDK the bridge type (‘NLNativeMultiBayConfig‘) still clamps the plan to 8, so an iOS session asked for 20 bays runs 8. Native 1.7.17 resolves it for the session’s mode, and the camera itself still falls back to 8 if the AR camera is unavailable (no ARKit / ARCore world tracking). An optional ‘guidanceStyle: ’classic’ | ’glance’‘ (case-insensitive) selects the guidance overlay style.

Attach free-text submit notes to the session before its payload uploads:

```
await Neurolabs.setMultiBaySubmitNotes('Aisle 4, promo end-cap', sessionId);
```

When a multi-bay session completes, the ‘cameraClosed‘ payload carries an extra ‘multiBay‘ summary (forwarded from the native SDK, the same shape on both platforms). The per-photo ‘captureQueued‘ / upload events are unchanged:

```
Neurolabs.addListener('cameraClosed', ({ sessionId, cancelled, captureCount, message, multiBa
y }) => {
  if (!multiBay) return; // single-bay session
  const {
    countsByLabel,        // { [detector class]: count } de-duplicated across bays
    baysCount,            // number of bays captured
    dedupTrusted,         // false if the cross-bay alignment chain broke
    previewImageFileUri,  // 'file://...' when generatePreview: true (optional)
    realogram,            // native 1.7.18: shelves -> rows -> slots (optional)
    segments              // native 1.7.18: [{ fromBay, toBay, trusted, countsByLabel }] (opt
ional)
  } = multiBay;
  console.log('multi-bay summary', baysCount, dedupTrusted, countsByLabel);
});
```

‘dedupTrusted: false‘ means the on-device de-duplication could not be fully trusted (the cross-bay alignment chain broke, the dedup timed out, or ‘onDeviceDedup‘ was disabled) - counts are then per-bay sums (an upper bound). Treat ‘countsByLabel‘ as an over-count in that case: the backend serves the raw per-image results unchanged (no server-side re-deduplication exists today - the uploaded alignment homographies make one possible later, but on-device dedup is currently the only cross-bay dedup).

‘countsByLabel‘ is keyed by detector class (‘sku_single‘, ‘sku_multipack‘, price tag, promo, poster), not by product or SKU.

From native SDK 1.7.18 the summary adds:

- ‘segments‘: the walk’s dedup segments. A pair of adjacent bays the SDK could not link starts a new segment, and the dedup runs inside each segment, so one broken link no longer leaves the rest of the walk unmerged. Each is ‘{ fromBay, toBay, trusted, countsByLabel }‘ (bays 0-based, inclusive). ‘dedupTrusted‘ is still true only for one trusted segment.
- ‘realogram‘: the walk’s stitched realogram (capture contract v1), its merged products grouped into shelves, rows and slots by geometry, so it exists with recognition off too. ‘{ version, coordinateSpace, sdkVersion, bays, shelves: [{ shelfIndex, rows: [{ rowIndex, slots: [...] }] }], truncated, omittedProductCount }‘; a slot is ‘{ slotIndex, xMin, yMin, xMax, yMax, bayIndex, detectorClass, trackId, productId?, stackIndex?, score? }‘. Shelf 0 is the top shelf, row 0 the row on the shelf board, slots run left to right, and coordinates are normalised to the first bay’s frame (later bays can read past 1). The same realogram goes up with the submit. It is absent when the merge timed out or failed. On Android the native result hands it over as a cache file; the plugin reads it off the main thread, puts it into ‘realogram‘ and deletes the file, so JS sees the same object on both platforms.

There is no partner-facing in-camera bay-mode switch: the Single Multi-bay toggle shown in Neurolabs demo builds is internal debug chrome (gated behind ‘NLSDKDebug.internalCameraToolsEnabled‘ on iOS and its Android equivalent, never enabled in partner builds). The capture mode is controlled purely by config: pass ‘multiBay‘ for a multi-bay session, omit it for single-bay.

6b. AR Multi-Bay Capture (native SDK v1.7.8+)

On the AR camera (‘cameraMode: ’ar’‘) one more option applies:

- ‘captureMode: ’guided’ | ’continuous’‘ (case-insensitive, default ‘’guided’‘). Guided keeps the shutter and shows side guidelines for the next frame; continuous captures on its own as the rep moves along the shelf, with a higher battery and thermal cost. Without a pose (a non-AR camera, or Android without Google Play Services for AR) the camera keeps the shutter and behaves as guided.

AR detection boxes are always world-locked: they lie on the shelf plane and stay put as the rep walks. There is no renderer option. ‘arBoxRenderer‘ was removed with the native legacy renderer, and a leftover ‘arBoxRenderer‘ key from older code is ignored on both platforms, not rejected.

AR overlay style (native SDK 1.7.16)

Opt-in looks for demos and partners; the rep defaults are unchanged. The values are the native SDKs’ wire values, case-insensitive. An unknown value is ignored and the default applies (it is never rejected), as the native SDKs decode it. They apply on the AR camera, with or without the preset.

- ‘arBoxStyle‘: ‘’quiet’‘ (default; alias ‘’standard’‘), ‘’identity’‘ (a colour per product) or ‘’highlight’‘ (every box a solid frame in ‘arBoxHighlightColor‘, default ‘#22C55E‘, slightly thicker, no fill).
- ‘arBoxUnmatchedColor‘ (native SDK 1.7.17): under ‘’highlight’‘, a box that has had a completed look and no match is drawn in this colour instead, default ‘#FACC15‘. It applies only with ‘arLiveLabels‘ on and the recognition provider ready; a box not looked at yet keeps ‘arBoxHighlightColor‘.
- ‘arCoverageStyle‘: ‘’tint’‘ (default), ‘’outline’‘ (a thin line in ‘arCoverageOutlineColor‘, default ‘#FACC15‘, drawn on the shelf plane around the captured bays) or ‘’tintAndOutline’‘ (alias ‘’tint_and_outline’‘). With no shelf-plane fit yet every style shows the tint.
- ‘arBoxPersistence‘: ‘’standard’‘ (default: a box fades about 18 s after it leaves view) or ‘’session’‘ (a settled box stays, dimmed, for the rest of the session; at most 300). Presentation only: counts are unchanged.
- Colours are ‘#RRGGBB‘ strings, forwarded as given; a malformed value draws the default colour.

AR box identity with recognition off (‘arAppearanceWithoutRecognition‘, native SDK 1.7.18)

- With on-device recognition off (‘init({ recognition: false })‘, no provider, or the remote kill switch), the AR boxes used to keep their identity from position alone, and touching facings of one product swapped identities. From native 1.7.18 they also use appearance: the ConvNeXt model the recognition stack uses, downloaded for appearance only for an organisation Operations grants it to, and a colour histogram until it is on the device or when it is not granted.

- ‘arAppearanceWithoutRecognition‘ (default ‘true‘) turns this on; ‘false‘ keeps position-only identity. Forwarded on both platforms, plain and under the ‘multiBayAR‘ preset. Presentation only: counts and uploads are unchanged.

High-resolution AR stills (Android, experimental, native SDK 1.7.18)

- ‘arHighResolutionStills: true‘ (default ‘false‘) runs ARCore on a Shared Camera session and takes each AR photo as a JPEG still of up to 4032 px (the 12 MP class) instead of ARCore’s CPU image (about 2.1 MP), then sizes it by ‘maxImageDimension‘ / ‘imageCompressionQuality‘ like every other photo. Forwarded plain and under the ‘multiBayAR‘ preset.
- The device decides whether a phone gets it: Android 10 (API 29) or later, a Shared Camera session ARCore accepts, a JPEG size larger than the CPU image, and the organisation’s remote switch (which can veto it). Any failure takes the CPU image for that capture, so a capture never fails because of it; two failed stills in a row switch it off for the session.
- **iOS accepts the key and ignores it.** iOS takes a full-resolution still on every AR capture already, so there is nothing to switch on.
- It stays off by default until the device checks on more Android phones pass. Try it on your target devices before turning it on for reps.

AR live labels and count (experimental, native SDK 1.7.16)

- ‘arLiveLabels: true‘ shows the top-1 product name on each stable box, at a match score of 0.6 or more, from the on-device recognition provider.
- Live labels need recognition on (no ‘recognition: false‘ at ‘init‘), the org’s catalog snapshot synced to the device (‘recognitionStatus‘ with ‘status: ’ready’‘) and an operations-capable API key, which the plugin uses to look up the names. With no provider (recognition off or not ready) there are no labels. When a requirement is missing, the plugin logs why at debug level.
- **Platforms:** Android shows the names. iOS shows them from native 1.7.17 on: native iOS 1.7.16’s SDK-managed camera takes no product-details source, so its boxes stay unnamed.
- ‘arLiveCount: true‘ shows a "~ N products" chip: the distinct boxes seen stable this session. It is an estimate; the review keeps the exact count.
- With recognition off (‘recognition: false‘) there are no names or unmatched colour, and a box is told apart from its neighbours by its position, not by which product it is: two identical facings that touch can be harder to keep apart. From native SDK 1.7.18 the boxes also use appearance in that case (on by default): a colour histogram, or the ConvNeXt model for an organisation Operations grants it to.
- The native ‘productDetailsSource‘ (the catalogue resolver behind the names) is an in-process object, so it is **not a Cordova option**. The plugin passes its own resolver, the one behind ‘getProductDetails(ids)‘.

```
// Live labels need recognition on and the catalog synced (see above).
await Neurolabs.init({ apiKey, recognition: true });
await Neurolabs.openCamera({
  preset: 'multiBayAR',
  arBoxStyle: 'highlight',
  arCoverageStyle: 'tintAndOutline',
  arBoxPersistence: 'session',
  arLiveLabels: true,
  arLiveCount: true,
});
```

For AR multi-bay, use the preset rather than assembling the options by hand:

```
// Guided (default): shutter + side guidelines, 1..4 bays
await Neurolabs.openCamera({ sessionId: crypto.randomUUID(), preset: 'multiBayAR' });

// Same preset with its inputs nested, here continuous and up to 6 bays
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  multiBayAR: {
    captureMode: 'continuous',
    multiBay: { minBays: 1, maxBays: 6 }
  }
});
```

The preset is the native SDKs' own 'NLNativeCaptureConfig.multiBayAR(...)' factory: AR camera, multi-bay on (1..4 bays unless you pass 'multiBay'), one review at the end, manual finish instead of auto-close, alignment guidance, glance guidance and detections metadata. Passing 'cameraMode: 'ar'' + 'multiBay' yourself does NOT reproduce it, because the plain 'openCamera' defaults (auto-close on, preview on) still apply.

How the AR multi-bay flow reviews bays (native SDK 1.7.15+):

- There is no per-bay "Snapshot preview". Each bay is committed at the shutter (a flash, a toast and a thumbnail), and the AR session keeps running between bays, so tracking and guidance carry on.
- "Review shelf" after Finish is the only review. Tapping a bay there opens it on its own: crop it, page with Previous / Next (or a swipe), and go back with "Back to review" on the last bay or the close button.
- While AR tracking is limited the guidance shows "Hold steady Finding your place", with no distance and no direction. If tracking comes back in a moved world, the old bay positions are dropped and the guidance says "Lost your place Continue from where you are".
- 'arFlowShowsCapturePreview: true' (default 'false') brings back the per-bay preview. It also needs 'showCapturePreview', and the AR session keeps running under the preview. Forwarded on both platforms.

Rules:

- 'preset: 'multiBayAR'' reads 'multiBay' and 'captureMode' from the top-level options; 'multiBayAR: {...}' reads them from the object. Giving either one in both places rejects with 'INVALID_ARGUMENT'.

- Any other option you set explicitly overrides that one field of the preset (e.g. ‘showAlignmentGuidance: false‘, ‘maxCaptures‘). Options you omit keep the preset’s value, not the plain ‘openCamera‘ default.
- ‘cameraMode: ’default’‘ with the preset rejects with ‘INVALID_ARGUMENT‘.
- ‘returnCaptureFileUris‘ stays whatever you set at ‘init()‘; the preset does not change it.
- Invalid ‘captureMode‘, ‘preset‘ or ‘multiBayAR‘ values reject with ‘INVALID_ARGUMENT‘ from the JS layer. The native bridges decode ‘captureMode‘ leniently, as the native SDKs do (unknown = guided), so a value can never fail a whole session on the native side.
- ‘getARCapabilities()‘ tells you, before opening the camera, whether AR capture will actually run (no ‘init()‘ needed):

```
const { worldTrackingSupported, depthSupported, installRequired } =
  await Neurolabs.getARCapabilities();
// iOS:     ARKit world tracking / LiDAR scene depth; installRequired is always false.
// Android: ARCore supported AND installed / ARCore Depth API; installRequired is
//          true when Google Play Services for AR is missing or too old (the AR
//          camera prompts for it, and falls back to the regular camera until then).
```

- On Android the plugin now declares ARCore (‘com.google.ar:core‘) 1.56.0, the version the SDK is built against. If your app pins ARCore itself, pin 1.56.0 or newer.

6c. Capture and quality checks only (no boxes, no recognition)

For hosts that use the SDK camera, its image quality checks, review and the upload queue, and want neither detection boxes on screen nor on-device recognition. The detector still runs once it is on the device, for the shelf guidance; it only downloads on Wi-Fi or Ethernet.

Config.

```
await Neurolabs.init({
  apiKey: '<API_KEY>',
  autoSyncEnabled: true,
  wifiOnlyUploadsEnabled: false,          // or the rep's own setting
  deletePhotosOnUploadSuccess: false,     // the host deletes with deletePhoto (see the queue
 note below)
  detectorPrefetchPolicy: 'unmetered',    // download the detector (~56 MB) only on Wi-Fi / E
thernet
  recognition: false,                     // never build on-device recognition
});

await Neurolabs.openCamera({
  type: 'shelf',
  cameraMode: 'default',
  guidanceMode: 'guidance',
  validationPreset: 'lenient',
  sessionId,
  taskUUID,
  confidenceThreshold: 0.25,
  iouThreshold: 0.45,
  showCapturePreview: false,
  maxCaptures: 10,
  showAlignmentGuidance: true,
  enableValidation: true,
  liveQualityChecksEnabled: true,
  liveQualityTargetFps: 6,
  showDetections: false,                  // no boxes; detection keeps running for the guidan
ce
  showCapturedRegions: false,
  sendDetectionsMetadata: true,
  enablePreviewCropping: true,
  allowManualFinish: true,
  autoCloseAfterCapture: false,
});
```

- ‘detectorPrefetchPolicy: 'unmetered'‘ needs native SDK 1.7.16. On a metered network (cellular, a metered or personal hotspot, or iOS Low Data Mode) the download waits and starts by itself once the device is on Wi-Fi or Ethernet, even mid-session. Until then ‘getDetectorModelStatus()‘ and ‘detectorModelStatusChanged‘ report ‘{ state: "not_downloaded", ready: false, reason: "waiting_for_unmetered_network" }‘. That reason is not a failure. Opening the camera on cellular does not download either. To download anyway, call ‘prefetchDetectorModel()‘; an explicit call ignores the policy. A detector already on the device is used on any network. Omitting the key means ‘'any'‘, and the plugin applies it on every ‘init‘.
- ‘recognition: false‘ stops the plugin from building the recognition bootstrap. There is no catalog snapshot or embedder download and no recognition network call, and ‘captureResult‘ carries no ‘recognitionMatches‘. ‘recognitionStatus‘ reports ‘{ status: "unavailable", reason: "disabled" }‘. A re-init with ‘false‘ retires a chain that an earlier init built.
- ‘showDetections: false‘ hides the detection boxes, and ‘showCapturedRegions: false‘ hides the captured-region overlay. Neither stops detection: once the detector is on the device, it runs every frame for the guidance below, and for ‘captureResult.detections‘ and the detection metadata. This holds on both platforms; on the AR camera (‘cameraMode: 'ar'‘) ‘showDetections: false‘ also hides the world-locked boxes.

- With both overlays off and no ‘validationPreset‘, both platforms use their lenient validation options (from native SDK 1.7.21). Before 1.7.21, Android used its stricter ‘IOS_PARITY‘ options in that case and iOS used lenient. An explicit ‘’default’‘, ‘’lenient’‘ or ‘’ios_parity’‘ was never affected. Before this plugin release, ‘’minimal’‘ was sent as no preset, so it was affected like an omitted preset; it now maps to the native minimal preset.
- ‘validationPreset: ’lenient’‘ applies the native SDKs’ lenient thresholds on both bridges. Plugin 1.6.6 mapped it to ‘’default’‘, so fewer captures are refused than with 1.6.6.

Quality checks before and after the detector is ready.

| When | Checks | |---|---| | From the first frame, detector or not (image-only) | blur, motion blur, exposure (dark / bright, exposure settling), glare, backlight, clipping, finger in frame, rotate the phone, perspective ("Face the shelf straight on", from the image and the device angle) | | Once the detector is on the device (it switches on mid-session, no reopen) | point at a shelf, move closer / step back / pull back, framing (frame more of the shelf, pan / tilt), row and product blur, level the camera, price tags (missing, blurry), cooler door |

While the detector is missing, the camera shows "Detector unavailable. You can still capture." (waiting for an unmetered network) or "Detector downloading". Capture, the shutter, review and the upload queue all work.

Readiness. ‘loadingProgress‘ reaches ‘step: "ready"‘ without the detector. ‘waitUntilReady()‘ resolves on it. **Never wait for ‘"model_ready"‘**: it never fires until the detector is on the device, and with ‘’unmetered’‘ that can be days for a rep who never uses Wi-Fi. None of the options above waits on the detector. To show whether detection is running, read ‘getDetectorModelStatus()‘ or listen to ‘detectorModelStatusChanged‘.

Upgrade checklist from plugin 1.6.6. These are the behaviour changes since 1.6.6 that a capture, quality, review, queue and photo-API host meets, with the plugin version that brought each one.

1. **No bundled detector (1.7.13).** The detector downloads at runtime, so the first launch needs a network for detection. Capture and uploads work without it. With ‘detectorPrefetchPolicy: ’unmetered’‘, that network must be Wi-Fi or Ethernet. 2. **‘loadingProgress‘ (1.7.13).** Without the detector, ‘loadingProgress‘ reaches ‘"ready"‘, never ‘"model_ready"‘. Replace any wait on ‘"model_ready"‘ with ‘waitUntilReady()‘ or a check for ‘"ready"‘. 3. **‘yoloModelVariant‘ (1.7.13).** ‘all_in_one‘ is the only detector. ‘nlb_model‘, ‘yolo_v26‘, ‘yolo_v11_seg‘, ‘shelf_rows‘ and ‘yolo11l_seg_sam‘ are still accepted, as deprecated aliases of it. An unknown value rejects ‘init‘ with ‘INVALID_ARGUMENT‘. 4. **‘customCameraTemplate‘ (1.7.10).** It must be an object. A JSON string (‘JSON.stringify(template)‘), number, array or boolean rejects ‘openCamera‘ with ‘INVALID_ARGUMENT‘. Each field is read on its own, so a bad field falls back to its default and the rest still applies. 5. **‘apiBaseURL‘ (1.7.9).** The key is ignored. Uploads go to the operations API; use ‘operationsBaseUrl‘ for staging. 6. **‘operationsBaseUrl‘ (1.7.13).** It is applied on every ‘init‘. An ‘init‘ without it resets the base to production. If you use staging, pass it on every ‘init‘. 7. **‘totalSizeBytes‘ (1.7.13).** In ‘getQueueStatus()‘ and ‘queueStatusChanged‘, it counts only the image storage of pending, uploading and failed items. Completed items you keep (‘deletePhotosOnUploadSuccess: false‘) are in ‘totalCount‘ but not in ‘totalSizeBytes‘. ‘getQueueDetails().totalSizeBytes‘ still sums every listed item. 8. **Boolean options (1.7.13).** Every boolean ‘init‘ option takes ‘true‘ / ‘false‘, a number (0 is false), or ‘"true"‘ / ‘"false"‘, ‘"1"‘ / ‘"0"‘, ‘"yes"‘ / ‘"no"‘, ‘"on"‘ / ‘"off"‘. Any other value keeps the default. On Android, ‘1‘, ‘"yes"‘ and ‘"on"‘ used to be ignored. 9. **String options (1.7.12).** On Android, a ‘null‘ or non-string option is treated as absent, as on iOS, and a non-string ‘apiKey‘ rejects ‘init‘. ‘getPhoto({ format })‘ rejects a non-string ‘format‘, and ‘setRetryPolicy(true)‘ rejects. 10. **iOS recognition frameworks (1.7.8).** The plugin always embeds the five recognition xcframeworks: ‘RecognitionInterface‘, ‘RecognitionEngine‘, ‘RecognitionEngineQdrant‘, ‘NLQdrantEdgeFFI‘ and ‘RecognitionBootstrap‘. They are linked even with ‘recognition: false‘, and they are dynamic, so keep them embedded. 11. **Android Kotlin metadata (1.6.8).** Stock Cordova Android apps compile against the Kotlin 2.3 SDK AAR without changes; the plugin injects ‘GradlePluginKotlinVersion 2.3.0‘. Remove any workaround you added for "binary version of its metadata is 2.3.0". The toolchain needs minSdk 26, compileSdk 36, AGP 8.9.1 or later, Gradle 8.11.1 or later (AGP 8.9.1’s minimum) and JDK 17. 12. **Events while the camera is open (1.6.8; the rate limit is from 1.7.8).** Capture events (‘captureQueued‘, ‘captureResult‘, ‘uploadSucceeded‘, ‘uploadFailed‘, ‘captureDeleted‘) are buffered while the native camera is on screen. They are delivered in order after ‘cameraClosed‘. ‘queueStatusChanged‘, ‘guidanceStateChanged‘ and ‘detectorModelStatusChanged‘ collapse to the latest value, and ‘queueStatusChanged‘ fires at most 4 times a second. If ‘cameraClosed.droppedEventCount‘ is above 0, re-read the queue. 13. **‘cameraClosed.error‘ (1.7.9).** A capture screen that failed (‘CAMERA_PERMISSION_DENIED‘, ‘CAMERA_UNAVAILABLE‘ on both platforms; ‘CAMERA_PRESENT_FAILED‘ on iOS; ‘NOT_INITIALIZED‘, ‘CAPTURE_RESULT_MISSING‘, ‘CAPTURE_RESULT_UNREADABLE‘ or ‘CAPTURE_FAILED‘ on Android) arrives as ‘cameraClosed‘ with ‘cancelled: false‘ and ‘error: { code, message }‘. It used to look like the rep cancelling. A refused permission and a camera that would not start are never ‘uploadFailed‘ (until 1.7.20 Android sent them there): when the camera showed the rep why (Android’s "Camera access is off" pane, the "camera could not start" message on both), the error arrives on that session’s ‘cameraClosed‘ once the rep closes it, provided nothing was captured. 14. **Portrait only (1.7.10).** Every capture reaches the server in portrait. 15. **Android ARCore (1.7.9).** The AAR is built against ARCore 1.56.0. 16. **Recognition at ‘init‘ (1.7.1 / 1.7.2).** Since 1.7.1, ‘init‘ builds the on-device recognition bootstrap whenever it has an operations key, which is always (the key falls back to ‘apiKey‘). The bootstrap asks the operations API whether the org has device recognition and, if it does, downloads the catalog snapshot and the embedder. Pass ‘recognition: false‘ to keep the 1.6.6 behaviour: no recognition at

Up to plugin 1.7.18 (native 1.7.18): - **Android.** The SDK never starts Sentry globally, and the plugin turns off ‘io.sentry.auto-init‘. Only errors the SDK captures itself are sent, through a private client. Crashes and ANRs in your app are never sent, and your own Sentry setup is untouched. ‘sentryEnabled: false‘ or ‘analyticsEnabled: false‘ at ‘init‘ stops all of it. - **iOS.** The plugin installs a process-wide Sentry crash handler when it loads, before ‘init‘ runs, so that a crash before ‘init‘ is still caught. That handler reports **every** crash in the app process to the Neurolabs Sentry project, including crashes in your own code. It also reports **app hangs and watchdog terminations**, because they are on by default in Sentry. Sentry’s global start is last-writer-wins: a Sentry your app started before the plugin loaded is replaced by this handler, and one your app starts afterwards replaces it. **‘sentryEnabled: false‘ at ‘init‘ does not stop this crash handler;** it only stops the SDK’s own error reports. To stop it, use the ‘config.xml‘ preference below (from plugin 1.7.19). Swizzling, session, network and performance tracking are off, and ‘sendDefaultPii‘ is off. - **Both platforms.** The SDK’s own error reports carry the error message as-is.

From plugin 1.7.19 (native 1.7.19), the policy is the same on iOS, Android and the plugin: - **Only the SDK’s own errors and crashes.** A filter sends only errors the SDK captures itself and crashes whose crashed thread runs SDK code. A crash in your app’s own code, an app hang, a watchdog termination or a failed request of your app is dropped. App-hang, watchdog and breadcrumb tracking are off. - **No credentials or personal data.** ‘sendDefaultPii‘ stays off, and messages are scrubbed: a URL keeps only its scheme, host and path. - **Opt out** with ‘sentryEnabled: false‘ (or ‘analyticsEnabled: false‘) at ‘init‘. On iOS this also closes the crash handler the plugin installed at load. It never closes a Sentry client your app started. - **Your app’s own Sentry is never replaced (iOS).** If your app starts Sentry before the plugin loads, the plugin starts no crash handler of its own, and the SDK reports only its own errors through its private client. If your app starts Sentry *after* the plugin loads, yours replaces the plugin’s crash handler (Sentry’s global start is last-writer-wins), and the SDK’s own errors still go through its private client. - **Known gap (iOS), without the ‘config.xml‘ preference below.** The crash handler is installed before ‘init‘. A crash report saved by an earlier run can therefore still be sent at the next launch, before ‘init‘ opts out, if it is a crash in SDK code. The preference closes this gap.

Silent from launch: the ‘config.xml‘ preference (from plugin 1.7.19). To turn the SDK’s Sentry off from app launch, before any JavaScript runs, add this to your ‘config.xml‘:

‘‘‘xml <preference name="NeurolabsSentryEnabled" value="false" /> ‘‘‘

- **Default: on.** Without the preference, Sentry behaves as described above. Only a clear false (‘false‘, ‘0‘ or ‘no‘, in any case) turns it off. Any other value leaves it on.
- **iOS.** The plugin skips its launch-time crash handler entirely. No Sentry client of ours exists from launch, so no crash report saved by an earlier run is sent. This closes the known gap above, and it also covers plugin 1.7.18’s host-crash, hang and watchdog reporting. - **Both platforms.** The preference wins over ‘init‘. The SDK’s own Sentry stays off whatever ‘init({ sentryEnabled })‘ says, because ‘init‘ cannot re-enable what you turned off at launch. Android starts no Sentry before ‘init‘, so on Android the preference only forces ‘sentryEnabled: false‘. - ‘analyticsEnabled‘ (Mixpanel) is not affected. Use ‘init({ analyticsEnabled: false })‘ for that. 23. **iOS background uploads and the app-delegate wake (plugin 1.7.19, native 1.7.19; added at runtime from plugin 1.7.20).** Queued photos upload through an iOS background ‘URLSession‘, so they keep going after iOS suspends or terminates the app and while the phone is locked. At launch the plugin adds ‘application(_:handleEventsForBackgroundURLSession:completionHandler:)‘ to ‘CDVAppDelegate‘, only if it has none and never replacing one, and hands the wake to the SDK. No ‘Info.plist‘ change and no native code are needed. If your ‘AppDelegate‘ already implements that method, or another plugin also handles it, forward to ‘NeurolabsSDKCore.handleEventsForBackgroundURLSession‘ from your ‘AppDelegate‘. In Swift, cordova-ios requires ‘override‘ there with or without this plugin. See section 5, "Uploads in the background and after the app is closed". 24. **Android toolchain check (plugin 1.7.20; real requirements from 1.7.21).** An Android hook stops the install or prepare early, with one message naming each unmet requirement and the plugin version, when the Android Gradle Plugin is older than 8.9.1, Gradle older than 8.11.1, compileSdk older than 36, minSdk older than 26, the Kotlin Gradle plugin older than 2.2 or the JDK older than 17. From 1.7.21 it no longer checks the cordova-android major: cordova-android 14 works once ‘config.xml‘ raises AGP, compileSdk and minSdk. From 1.7.21 it runs before the install, so a failed check installs nothing. Plugin 1.7.20 rejected cordova-android 14 even then (on 1.7.20 only, ‘NEUROLABS_SKIP_ANDROID_TOOLCHAIN_CHECK=1‘ gets past it), and after a failed install there, run ‘cordova plugin rm ai.neurolabs.cordova‘ before retrying. See section 3.

6d. Visit KPIs and store context (native SDK 1.7.21)

When a capture session finishes with at least one photo, the SDK computes the visit’s KPIs and next best actions on the device and the plugin emits ‘visitKpis‘. KPIs are measured against the outlet’s **store context**: its targets, assortment and planograms, plus the org’s metric definitions. The SDK downloads it through the Operations API, checks its signature against a key pinned in the SDK, and keeps it on the device for the org’s offline TTL, so a visit computes KPIs offline. The calls, arguments, payloads and error messages are the same on iOS and Android.

1. Cache the route. At sign-in or on Wi-Fi, fetch the store context for the outlets the rep will visit. It runs on any network, in batches of 50.

```
const { statuses, refreshFailure } = await Neurolabs.prefetchStoreContext(routeOutletIds);
for (const [outletId, status] of Object.entries(statuses)) {
  if (status.state !== ’ready’) console.warn(’no store targets for’, outletId, status.reason)
;
}
```

It never rejects on network or state. ‘refreshFailure‘ (absent when every fetch was applied) is ‘no_credentials‘, ‘no_offline_identity‘, ‘no_valid_outlet_ids‘, ‘denied‘, ‘unavailable‘, ‘server_error‘ or ‘bad_request‘. The cache is kept in every case except ‘denied‘.

2. Tell the SDK which outlet the visit is at. After ‘openCamera‘ resolves (or pass your own ‘sessionId‘ to ‘openCamera‘ and call this first):

```
const { sessionId } = await Neurolabs.openCamera({ type: 'shelf' });
await Neurolabs.setVisitKpiContext(sessionId, {
  outletId: visit.outletId,            // Operations outlet UUID (GET /v1/outlets)
  externalVisitId: visit.id,           // your visit id, 1-128 characters
  sectionsByCaptureId: {},             // capture id  outlet subsection id (beats the bay's t
ag)
  sectionsByBayIndex: { 1: 'drinks' }, // 1-based bay  outlet subsection id
});
```

It also starts a background refresh of that outlet’s store context when it was not confirmed online within the last hour; the session never waits for it. Calling it again replaces the session’s context. Without it, the SDK uses the open collector mission’s outlet from ‘/v1/config‘, if there is one, and every KPI row covers the whole visit. It needs ‘init()‘ (‘NOT_INITIALIZED‘). A malformed context rejects with ‘INVALID_ARGUMENT‘.

3. Show the result.

```
Neurolabs.addListener('visitKpis', (event) => {
  // { sessionId, provisional, storeContext, offline, willSubmit, actionCount,
  //   perfectStoreScoreBp?, payloadJson, payload }
  if (event.storeContext === 'used') {
    showKpis(event.payload, { label: 'Provisional', score: event.perfectStoreScoreBp });
  } else {
    showGapsAndPriceTags(event.payload, { banner: 'Needs store targets' });
  }
});
```

- ‘payload‘ is the ‘nl.visit_kpis‘ v1 object; ‘payloadJson‘ is the same thing as the canonical JSON text.
- ‘provisional‘ is always ‘true‘ on the device. Label the result "Provisional" until the backend’s recomputed KPIs replace it.
- ‘storeContext‘: ‘used‘ (the outlet’s targets applied; ‘offline‘ is ‘true‘ when the cache was not confirmed online within the hour), ‘missing‘ or ‘expired‘. Without a store context the payload has gaps and price-tag presence only, and no Perfect Store score: show "Needs store targets", and ask ‘getStoreContextStatus(outletId)‘ why. Treat a value you do not know as ‘missing‘.
- ‘perfectStoreScoreBp‘: the Perfect Store score in basis points (0-10000); absent when none was computed.
- ‘willSubmit‘: these exact bytes ride the session’s mission submit as ‘client_metadata.device_kpis‘. That happens only with a ready, signature-verified store context and a payload that passes the backend’s checks. Otherwise the result stays on the device.

- One event per session that finished with photos, not cancelled, while visit KPIs are on. It is buffered while the camera is open and replayed after that session's ‘cameraClosed‘. It never fires when the computation fails; the SDK reports that failure itself.

Status of one outlet. ‘getStoreContextStatus(outletId)‘ reads the cache only:

| Field | Meaning | |---|---| | ‘state‘ | ‘ready‘ (the next visit uses it), ‘expired‘ or ‘missing‘ | | ‘reason‘ | Absent when ‘ready‘. ‘expired‘: ‘ttl‘, ‘valid_until‘, ‘clock_skew‘. ‘missing‘: ‘no_outlet‘, ‘not_assigned‘ (the outlet is not your org's: check the id), ‘no_context‘ (no live mission or nothing published), ‘not_cached‘, ‘org_part_unavailable‘, ‘no_credentials‘, ‘no_offline_identity‘, or a refused body's code (‘hash_mismatch‘, ‘invalid_signature‘, ...). Treat an unknown code as missing. | | ‘offline‘ | ‘ready‘ but not confirmed online within the last hour | | ‘signatureVerified‘ | ‘ready‘ and signature-checked against a pinned key: the only state whose KPIs are submitted | | ‘confirmedAt‘, ‘validUntil‘, ‘usableUntil‘ | ISO-8601 UTC in whole seconds. ‘usableUntil‘ (ready only) is when it stops being usable offline. |

Other calls.

- ‘setVisitKpisEnabled(false)‘ turns visit KPIs off: no computation, no ‘visitKpis‘, nothing added to the submit, no background refresh. On by default.
- ‘clearStoreContextCache()‘ deletes every cached store context on the device (sign-out).
- ‘setStoreContextIdentity(identity)‘ names the cache after the signed-in rep. It only matters when the operations credential comes from a host auth provider; with the API key this plugin uses, the key names the cache and the SDK ignores the identity.
- The store-context calls and ‘setVisitKpisEnabled‘ work before ‘init()‘: the reads resolve ‘missing‘ / ‘no_credentials‘, a clear runs when ‘init()‘ installs the SDK, and the switch and the identity hold across re-inits.

7. Post-Processing + Lifecycle Events

```
// Fires when the camera screen is dismissed (Done or Close pressed).
// captureCount is the number of photos queued for upload - NOT the photo data itself.
// To retrieve photo data, use getPhoto() with the captureId from captureQueued (see section
8).
Neurolabs.addListener('cameraClosed', ({ sessionId, cancelled, captureCount, message, dropped
EventCount, error }) => {
  console.log('cameraClosed', sessionId, cancelled, captureCount, message);
  // The capture screen failed (e.g. camera permission denied, SDK not
  // initialized, unreadable session result). 'cancelled' is false; show
  // 'error.message' and branch on 'error.code'.
  if (error) showCaptureError(error.code, error.message);
  // 0 on every ordinary session. Non-zero means the deferred-event buffer
  // overran and the replay that follows is incomplete - re-read the queue.
  if (droppedEventCount > 0) Neurolabs.getQueueStatus().then(console.log);
});

// Fires once per photo taken, immediately after each capture is added to the upload queue.
// captureQueued events are buffered while the camera is open and always delivered after came
raClosed.
// Use the captureId here to retrieve the local photo via getPhoto().
// imageFileUri ('file://...') is present ONLY when file-URI streaming is enabled
// via init({ returnCaptureFileUris: true }) - see 8.1.
Neurolabs.addListener('captureQueued', ({ captureId, queueItemId, sessionId, imageFileUri })
=> {
  console.log('queued', captureId, queueItemId, imageFileUri);
});

// Real-time capture guidance feedback (buffered + coalesced like captureQueued).
// Payload: { rule, tier, timestamp, byDegrees? } where rule is one of
// DISTANCE_TOO_CLOSE | DISTANCE_TOO_FAR | ANGLE_OFF | PANNING_TOO_FAST.
Neurolabs.addListener('guidanceStateChanged', (payload) => console.log('guidance', payload));

// Fires once per photo with what the SDK saw on the device (buffered like captureQueued).
// Payload: { captureId, sessionId, success, message, detectionCount, capturedRegionCount,
//            detections, recognitionMatches?, captureMetadata?, quality? }
// captureMetadata (native SDK 1.7.18) is the capture's metadata record (see 4a).
// It can carry the device's location: never log it.
Neurolabs.addListener('captureResult', ({ captureId, success, detectionCount }) => {
  console.log('captureResult', captureId, success, detectionCount);
});

Neurolabs.addListener('uploadSucceeded', (item) => console.log('uploaded', item));
Neurolabs.addListener('uploadFailed', (payload) => console.warn('uploadFailed', payload));

// QUEUE_FULL and STORAGE_LOW (native SDK 1.7.16: under 500 MB free) refuse a
// NEW photo. From native 1.7.21 the camera asks the queue before the shutter,
// so the photo is never taken: this event has no captureId, and there is no
// captureResult or captureQueued for it. The camera shows the reason until a
// press finds room. STORAGE_LOW is retryable once space is freed.
Neurolabs.addListener('uploadFailed', ({ errorCode }) => {
  if (errorCode === 'STORAGE_LOW') askRepToFreeSpace();
  else if (errorCode === 'QUEUE_FULL') Neurolabs.flushQueue();
});

// Kept uploaded photos reached 80% of maxRetainedPhotos (see 5).
// Payload: { count, bytes, oldestCreatedAt?, limit }
Neurolabs.addListener('retainedPhotosHigh', (payload) => console.warn('retainedPhotosHigh', p
ayload));
Neurolabs.addListener('queueStatusChanged', (status) => console.log('queueStatusChanged', sta
tus));
```

8. Retrieving Photos Locally

Photos are stored in the device's app cache after capture. Use 'getPhoto()' to access them by 'captureId' (from 'captureQueued') or 'queueItemId'.

```
const capturedIds = [];

Neurolabs.addListener('captureQueued', ({ captureId }) => {
  capturedIds.push(captureId);
});

Neurolabs.addListener('cameraClosed', async ({ cancelled }) => {
  if (cancelled) return;
  for (const captureId of capturedIds) {
    // format: 'fileUri' returns { uri: 'file://...' } - a temp path in the app cache
    // format: 'base64'  returns { base64: '...' }    - requires allowBase64PhotoExport: true
 in init()
    const photo = await Neurolabs.getPhoto({ captureId, format: 'fileUri' });
    console.log('photo uri', photo.uri);
  }
  capturedIds.length = 0;
});
```

'deletePhoto()' accepts the same query shape ('captureId', 'queueItemId', or 'responseId') and removes the local file from the cache.

> In file-URI streaming mode (8.1) 'getPhoto({ format: 'base64' })' rejects with > 'UNSUPPORTED_FORMAT' - base64 export is forced off. Use 'format: 'fileUri'', or read the > 'imageFileUri' handed to you on 'captureQueued' directly.

8.1 Capture File-URI Streaming (v1.6.5, OOM fix)

Large multi-capture sessions (wide shelves, dozens of full-resolution JPEGs) could previously push peak memory high enough to get the app OOM-killed, because every captured image was held resident until the host consumed it.

Enable **file-URI streaming** to avoid this - an app-wide, opt-in toggle set once at 'init()':

```
await Neurolabs.init({ apiKey: '<API_KEY>', operationsApiKey: '<OPS_KEY>', returnCaptureFileU
ris: true });
```

Default is 'false' (unchanged behavior). When 'true':

- Each capture is written to a **stable, backup-excluded on-disk store** and handed to the host as 'imageFileUri' on the 'captureQueued' event, instead of being held in memory. Peak heap no longer grows with the number of captures.
- **base64 export is forced off**: 'getPhoto({ format: 'base64' })' rejects with 'UNSUPPORTED_FORMAT'. Read the 'imageFileUri' instead.
- The retained files **outlive the capture session** - they are kept until the host explicitly releases them, so you can stream/upload them after 'cameraClosed'.

The flag is honored on **both platforms**: Android bakes it into the init-level native configuration; iOS applies it to every capture session.

Releasing retained files

Because the files persist past capture-finish, the host **must** release them once it has finished streaming/uploading each ‘imageFileUri‘, otherwise they linger until a 72-hour TTL sweep reclaims them. Two APIs:

```
const queued = [];
Neurolabs.addListener('captureQueued', ({ captureId, sessionId, imageFileUri }) => {
  queued.push({ captureId, imageFileUri });
});

Neurolabs.addListener('cameraClosed', async ({ sessionId, cancelled }) => {
  if (cancelled) return;
  for (const { captureId, imageFileUri } of queued) {
    await uploadFromFileUri(imageFileUri);        // your host-side upload
    await Neurolabs.releaseCapture(captureId);    // per-capture release
  }
  queued.length = 0;

  // ...or release the whole session at once instead of per capture:
  // await Neurolabs.acknowledgeCaptures(sessionId);
});
```

- ‘acknowledgeCaptures(sessionId)‘ - bulk release: deletes every retained file for the session. Missing/blank ‘sessionId‘ rejects with ‘INVALID_ARGUMENT‘.
- ‘releaseCapture(captureId)‘ - single-capture release, for hosts that upload incrementally. Missing/blank ‘captureId‘ rejects with ‘INVALID_ARGUMENT‘.

Both are no-ops for sessions/captures that kept no file (legacy byte-return mode, or already released), and neither touches the SDK’s own upload queue (a separate disk-backed store that streams its uploads independently).

9. Notes

- ‘openCamera‘ is the custom native camera entrypoint.
- Use the strict shelf payload above for full shelf guidance checks and auto-close after a validated save.
- ‘liveQualityChecksEnabled=true‘ is required for pill/rotation/warning/error guidance behavior.
- Keep ‘type: ’shelf’‘, ‘guidanceMode: ’strict’‘, ‘showDetections: false‘, and ‘showCapturedRegions: false‘ for the custom-camera guidance UI path.
- ‘guidanceMode: ’guidance’‘ should only be used as a temporary fallback if a partner explicitly wants looser Android behavior than the parity recommendation.
- ‘autoCloseAfterCapture=true‘ closes after Save from preview/review flow.
- ‘Done‘ appears only in manual-finish mode: ‘autoCloseAfterCapture=false‘ + ‘allowManualFinish=true‘.
- Use ‘minCapturesBeforeDone‘ to require a minimum number of captures before ‘Done‘ becomes active.
- If ‘allowManualFinish=false‘, ‘maxCaptures‘ acts as a hard limit and capture disables at the limit.

- ‘captureQueued‘ events are buffered while the camera is open and always delivered after ‘cameraClosed‘ - never during an active session. The buffer holds 512 entries; ‘cameraClosed‘’s ‘droppedEventCount‘ reports how many it had to discard (0 unless a pathological session overran it).
- Keep base64 transport disabled unless explicitly needed.

Troubleshooting: "Unresolved reference" Kotlin errors on Android build

Symptoms like ‘Unresolved reference ’setOperationsBaseUrl’‘, ‘’setMultiBaySubmitNotes’‘, ‘’resourceName’‘, or ‘’labelsResourceName’‘ during the Android build mean the plugin source is being compiled against a **stale ‘neurolabs-android-sdk.aar‘** - an old AAR left in ‘platforms/android/app/libs/‘ (which takes precedence) from a previous plugin version.

Fix:

1. Delete the stale AAR and its metadata: ‘platforms/android/app/libs/neurolabs-android-sdk.aar‘ (+ ‘.meta.json‘) 2. Re-run ‘cordova prepare android‘ so the plugin re-fetches the AAR matching the plugin’s pinned native version. 3. Rebuild.

Since v1.6.3 the Android Gradle script fails the build early with an explicit version-mismatch message (instead of the confusing Kotlin errors) whenever the resolved AAR’s version does not match the plugin’s ‘neurolabs.sdkVersions.android‘.

Upgrading from a pre-1.6 plugin - queue durability

On pre-1.6 native SDKs the upload queue was wiped by the very bridge message that carries your API config. Sending ‘NL_Config‘ / the ‘api_config‘ action (which you do to set credentials + per-visit ‘accountId‘/‘storeId‘/‘visitId‘) triggered a full queue clear on the native side - metadata **and** the stored JPEGs - so any not-yet-uploaded captures from a previous visit were deleted at the start of the next one, regardless of network.

- **Already-lost captures cannot be recovered.** They were removed from device storage by the reconfigure-wipe - even sessions that were never lost to timeouts were destroyed this way. There is nothing to migrate.
- **Upgrading stops the loss going forward.** From native v1.6.1+ (plugin bundling native 1.6.1) the config message no longer clears the queue; captures persist across reconfigures, backgrounding, and restarts. Only an explicit ‘clearUploadQueue()‘ removes them.

Upgrade to a plugin bundling native **1.6.1** (1.6.3+ recommended) and set ‘operationsApiKey‘ at init so the preserved queue can deliver.

On-Device Recognition + Tap-to-Product-Card (v1.7.1, Android)

Config-gated per account: your operations API key resolves your org’s recognition config and vector dataset - enabling it is a backend flip, no app change. When active, ‘captureResult‘ events carry ‘recognitionMatches: { <detectionId>: { catalogItemId, similarity } }‘ (each detection object carries the correlating ‘id‘); resolve them for your card UI:

```
const details = await Neurolabs.getProductDetails(
  Object.values(payload.recognitionMatches).map((m) => m.catalogItemId)
);
// details["<uuid>"]  { canonicalUuid, name?, brand?, flavour?,
//                      containerSize?, barcode?, thumbnailUrl? }
```

Absent keys are unknown products - render the raw UUID + similarity as the fallback. Results are cached natively for 24 h. iOS parity for the recognition provider arrives in a follow-up release; ‘getProductDetails‘ works on both platforms today.